DEVELOPER PROJECT REPORT

PDF Editor

A browser-based PDF editor, and an honest look at how it's built

Repository: quantiradev/qt_pdf-editor · Branch: sprint1
Report Date: July 13, 2026
Classification: Internal · Engineering Documentation

- The REST API is broad and consistent, covering the entire PDF lifecycle · CRUD, page operations, merge/split, form fields, watermarking, and multi-format export.

Where the Risk Sits

- No automated test suite yet · no unit tests, no integration tests, nothing end-to-end.
- Deployment today means running a Windows batch script that starts Uvicorn locally; there's no Dockerfile and no CI/CD pipeline.
- auth.py falls back to a hardcoded JWT secret if the environment variable isn't set, and passwords are hashed with only 1,000 PBKDF2 iterations · both worth fixing before this goes anywhere near the public internet.
- Metadata lives in a single db.json file guarded by an in-process lock, which is fine for one server and won't survive scaling out to more than one.

2. Project Overview & Objectives

2.1 The Problem

Editing a PDF properly usually means opening a desktop app like Acrobat, or routing the file through some conversion pipeline. QT PDF Editor's whole premise is to skip that: upload once, and everything after · viewing, marking up, restructuring · happens against that same file, in the browser, with edits written straight back into a standards-compliant PDF rather than bounced through an intermediate format.

2.2 What It Sets Out To Do

- Render PDFs faithfully in the browser, fast enough that scrolling and zooming actually feel responsive.
- Cover the annotation types people actually reach for: text, highlight/underline/strikeout, freehand ink, shapes, images, sticky notes, hyperlinks.
- Support real in-place text editing rather than an overlay box pretending to be one, by detecting paragraph structure and reflowing edited text with the source document's own fonts.
- Handle document management, not just markup: rename, soft/hard delete, restore, rotate/insert/duplicate/reorder/delete/extract pages, split, merge.
- Gate access behind accounts, and offer export to PDF, PNG, or JPEG with control over page range and DPI.
- Keep a real undo/redo history at the document level, so a bad edit doesn't mean starting over.

2.3 Who It's For

Individuals and small teams who need to mark up, redline, fill in, or lightly restructure a PDF, and would rather not install anything to do it.

3. System Architecture

The system splits along the usual client/server line, and it sticks to that split cleanly: the client owns rendering and interaction, the server owns the actual PDF bytes and every structural change made to them. Nothing that affects document correctness is duplicated on the client · if you want to know what a document actually looks like, the server's copy is the answer.

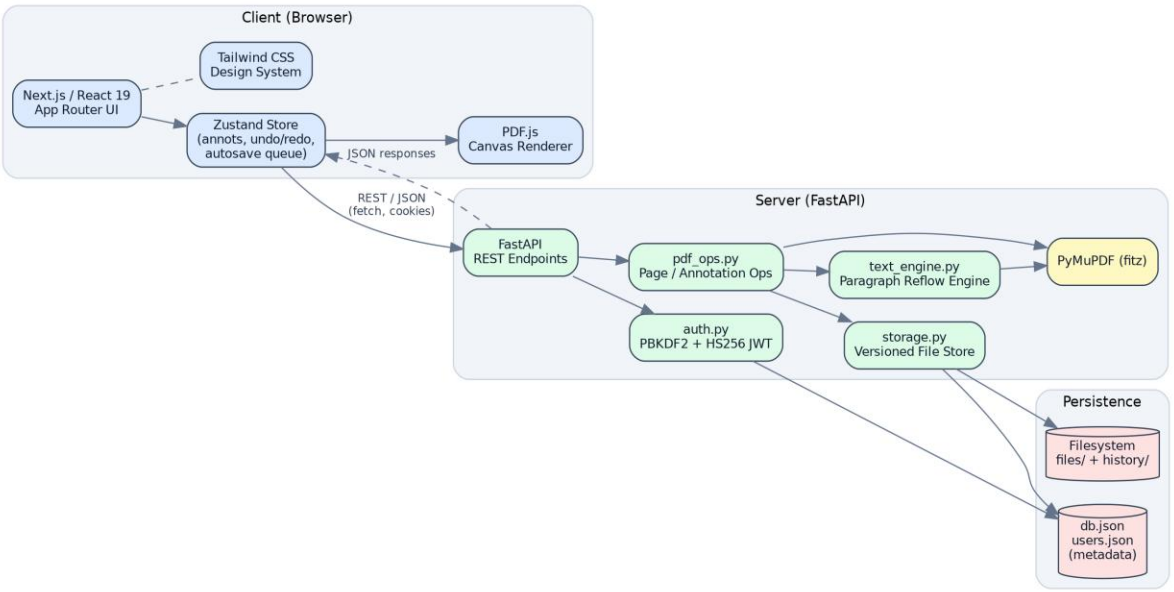

Figure 1 – QT PDF Editor system architecture

3.1 Frontend

Next.js (React 19, App Router)	Routing for auth, editor, preview, and profile pages; the editor itself lives in client components
Zustand	Holds the editor's live state: the loaded document, annotation list, undo/redo stacks, tool options, panel layout, autosave queue
Tailwind CSS v4	Styling across the whole app, from the landing page to the editor chrome
PDF.js (pdfjs-dist)	Renders pages to <canvas> on the client · the only place the PDF actually gets parsed for display
Lucide React	The icon set used throughout the toolbar and sidebars

The editor itself (components/editor-studio/) is built around one Editor.tsx shell that pulls together a Toolbar, a LeftSidebar for pages and the outline, CanvasStage/PageView for the actual rendering, a RightSidebar for properties and the annotation list, an AnnotLayer overlay that handles interactive editing, and a StatusBar. A separate Previewer/PreviewTopBar pair handles the read-only view used for shared documents.

3.2 Backend

FastAPI	The REST layer · routing, request validation via Pydantic, structured error handling

Technology	Role
Uvicorn	The ASGI server the FastAPI app runs on (directly, or via `python main.py --port`)
PyMuPDF (fitz)	Does the real work: all parsing, rendering-to-image, and mutation · annotations, pages, forms, redaction
fonttools	Used by the reflow engine to inspect embedded fonts, so it can decide whether to reuse one or fall back
python-multipart	Handles the multipart upload on `/api/files/upload`

Four modules do essentially all of the backend's work: main.py holds every HTTP route, auth.py owns the user store and JWT sign/verify logic, storage.py is the file-backed metadata store with per-file locking and undo/redo snapshotting, pdf_ops.py handles page- and annotation-level PDF operations, and text_engine.py is the paragraph detection and reflow engine behind in-place text edits.

3.3 Data & Storage Layer

There's no database here in the traditional sense. File metadata lives in backend/data/db.json, keyed by a 12-character hex id, with the PDF bytes sitting next to it in backend/data/files/{id}.pdf. Every mutating operation copies the pre-change PDF into backend/data/history/ before touching it, and storage.py keeps a capped, per-file undo/redo stack (20 entries) pointing at those snapshots. User accounts follow the same pattern in a flat users.json.

- A single module-level threading.Lock() serializes reads and writes of db.json, since FastAPI's sync endpoints run on a thread pool and can genuinely overlap.
- A second, per-file lock (storage.op_lock) additionally serializes concurrent edits against the same document, so two requests can't race on one PDF.
- That's a sound design for a single process, but it doesn't extend to multiple server instances · more on that in Sections 7 and 10.

4. Core Workflow: The Annotation Bake Pipeline

If there's one piece of this system worth understanding properly, it's how an edit made in the browser turns into a permanent, structurally correct change in the actual PDF. The team's own term for this is "baking" an edit, and it's a genuinely careful piece of engineering.

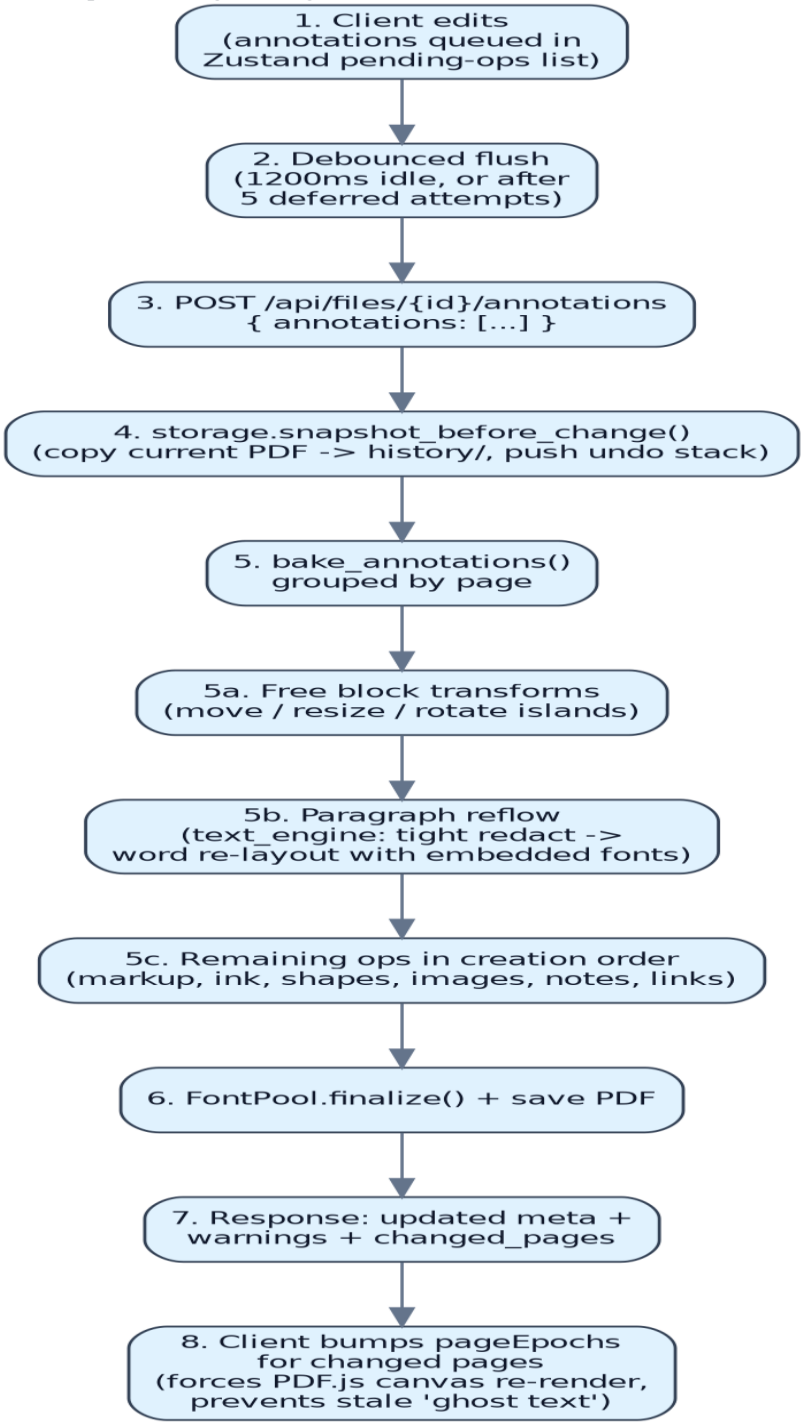

Figure 2 – Client edit to persisted PDF: the annotation bake sequence

4.1 Why Ordering Matters Here

bake_annotations() in pdf_ops.py doesn't just apply a batch of pending edits in the order they arrived · it processes them in three deliberate passes, because redaction is destructive to whatever's sitting in that page region at the time:

- Pass 1, free block transforms: text blocks the user has picked up and moved, resized, or rotated as floating islands. These redact their old spot and redraw somewhere else, so they need to happen before anything else touches the page.
- Pass 2, paragraph reflow: in-place text rewrites go next, because their redaction step would otherwise wipe out any annotation the user had visually placed over that same paragraph.
- Pass 3, everything else: highlights, ink, shapes, images, notes, links · applied in whatever order they were originally created.

That ordering is exactly what prevents the "ghost text" problem the team ran into during development. If a text edit's redaction ran after something had already been drawn on top of it · or if the client didn't force the canvas to actually re-render after the server rewrote the page · old glyph data could linger on screen until the next full repaint. The fix has two halves: the server-side pass ordering described above, and a client-side counterpart where the frontend bumps a per-page "epoch" counter (pageEpochs, in the Zustand store) for every page the server reports as changed. That forces PDF.js to throw away the old canvas bitmap for those pages instead of quietly compositing over it.

5. Notable Engineering Challenges & Resolutions

5.1 The Ghost Text Race Condition

Covered in detail in Section 4.1. The short version: a strict three-pass ordering on the server means redactions never wipe out content drawn after them, and a client-side render-epoch bump makes sure PDF.js discards any stale canvas for a page instead of layering new content over old.

5.2 Redaction Mask Rendering

The naive approach · redact a paragraph's whole bounding box before rewriting it · tends to bleed into the lines above and below, or leave faint edge artifacts. text_engine.py avoids this by computing a tight redaction rectangle for each individual visual line (_line_rect), shaving a small inset off the top and bottom of each line (_REDACT_INSET_V, 0.75pt, capped at 18% of line height) so redaction never touches a neighboring line. It then calls PyMuPDF's apply_redactions() with flags that preserve images and line art, so nothing outside the text itself gets disturbed. One wrinkle: redaction rebuilds the page's content stream and drops its font resources in the process, so the FontPool object deliberately forgets any font aliases it had registered for that page right after redaction runs (page_redacted()) · otherwise text reinserted afterward would point at font resources that no longer exist.

5.3 Font Family Detection & Reuse

When someone edits an existing paragraph, the goal is to keep using the document's own embedded font rather than quietly swapping in a generic PDF base-14 font, so the edit doesn't visually stick out. A FontPool, built fresh for each bake operation, checks · via fonttools · whether the embedded font's glyph set actually covers the characters in the new text before reusing it. If it doesn't, it falls back to the closest base-14 family (helv/tiro/cour, in regular, bold, italic, or bold-italic) instead.